

SoGoodDAOFactory.sol

SoGoodDAOFactory.sol

Purpose:

A local governance as a service —
powered by Sotility's own SoGood
Social platform

Its an on-chain DAO deployer —
allowing any community to spin up their
own governance with ease.

This factory contract allows any
verified business or community to
deploy its own DAO using a cloned
ProposalManager.
Perfect for local projects, housing
co-ops, or worker-owned collectives.

Features:

Deploy ProposalManager with custom
token/quorum

Link it to a business or IPFS registry

Emits event for off-chain dashboards

Uses minimal clone pattern for gas
efficiency

What it Does:

Lets any admin-approved creator
spawn their own ProposalManager

Saves the DAO metadata on-chain

Emits clean events for indexing or
frontend feeds

```
---

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts
/access/AccessControl.sol";
import "./ProposalManager.sol";

/**
 * @title SoGoodDAOFactory
 * @dev Deploys local
ProposalManager DAOs for
community-run businesses or causes.
 */
contract SoGoodDAOFactory is
AccessControl {
    bytes32 public constant
CREATOR_ROLE =
keccak256("CREATOR_ROLE");

    address[] public deployedDAOs;

    struct DAORecord {
        address creator;
        address dao;
        address votingToken;
        string ipfsMetadata;
        uint256 timestamp;
    }

    DAORecord[] public daoRegistry;

    event DAOCreated(address
indexed dao, address token, string
ipfsMetadata);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE,
msg.sender);
        _grantRole(CREATOR_ROLE,
msg.sender);
    }

    function createDAO(
        address votingToken,
        uint256 voteDuration,
        uint256 quorumThreshold,
        string calldata ipfsMetadata
    ) external onlyRole(CREATOR_ROLE)
returns (address) {
        ProposalManager newDAO =
new ProposalManager(votingToken,
voteDuration, quorumThreshold);
        deployedDAOs.push(address(newD
AO));

        daoRegistry.push(DAORecord({
            creator: msg.sender,
            dao: address(newDAO),
            votingToken: votingToken,
            ipfsMetadata: ipfsMetadata,
            timestamp: block.timestamp
        }));

        emit
DAOCreated(address(newDAO),
votingToken, ipfsMetadata);
        return address(newDAO);
    }

    function getAllDAOs() external view
returns (DAORecord[] memory) {
        return daoRegistry;
    }
}
```